Notes:

- HW2 graded, solutions posted
- HW3 Due October 28
- HW4 posted

Last class:

- Using venvs to manage dependencies
- Intro to pandas and dataframes
- reading data
- basic statistics
- slicing and masking

In [2]:
```python
import pandas as pd
data_americas = pd.read_csv('./data/gapminder_gdp_americas.csv', usecols=range(1,14),index_col="country")
```

Let's look at a more complicated example:

In [3]:
```python
print(data_americas.mean())
```

```
gdpPercap_1952     4079.062552
gdpPercap_1957     4616.043733
gdpPercap_1962     4901.541870
gdpPercap_1967     5668.253496
gdpPercap_1972     6491.334139
gdpPercap_1977     7352.007126
gdpPercap_1982     7506.737088
gdpPercap_1987     7793.400261
gdpPercap_1992     8044.934406
gdpPercap_1997     8889.300863
gdpPercap_2002     9287.677107
gdpPercap_2007    11003.031625
dtype: float64
```

This gives mean GDP per capita in each year.

In [4]:
```python
mask_higher = data_americas > data_americas.mean()
print(mask_higher)
```

```
                    gdpPercap_1952  gdpPercap_1957  gdpPercap_1962  \
country
Argentina                     True            True            True
Bolivia                      False           False           False
Brazil                       False           False           False
Canada                        True            True            True
Chile                        False           False           False
Colombia                     False           False           False
Costa Rica                   False           False           False
Cuba                          True            True            True
Dominican Republic           False           False           False
Ecuador                      False           False           False
El Salvador                  False           False           False
Guatemala                    False           False           False
Haiti                        False           False           False
Honduras                     False           False           False
Jamaica                      False            True            True
Mexico                       False           False           False
Nicaragua                    False           False           False
Panama                       False           False           False
Paraguay                     False           False           False
Peru                         False           False            True
Puerto Rico                  False           False            True
Trinidad and Tobago          False           False            True
United States                 True            True            True
Uruguay                       True            True            True
Venezuela                     True            True            True

                    gdpPercap_1967  gdpPercap_1972  gdpPercap_1977  \
country
Argentina                     True            True            True
Bolivia                      False           False           False
Brazil                       False           False           False
Canada                        True            True            True
Chile                        False           False           False
Colombia                     False           False           False
Costa Rica                   False           False           False
Cuba                          True           False           False
Dominican Republic           False           False           False
Ecuador                      False           False           False
El Salvador                  False           False           False
Guatemala                    False           False           False
Haiti                        False           False           False
Honduras                     False           False           False
Jamaica                       True            True           False
Mexico                        True            True            True
Nicaragua                    False           False           False
Panama                       False           False           False
Paraguay                     False           False           False
Peru                          True           False           False
Puerto Rico                   True            True            True
Trinidad and Tobago          False            True            True
United States                 True            True            True
Uruguay                      False           False           False
Venezuela                     True            True            True

                    gdpPercap_1982  gdpPercap_1987  gdpPercap_1992  \
country
Argentina                     True            True            True
Bolivia                      False           False           False
Brazil                       False            True           False
Canada                        True            True            True
Chile                        False           False           False
Colombia                     False           False           False
Costa Rica                   False           False           False
Cuba                         False           False           False
Dominican Republic           False           False           False
Ecuador                      False           False           False
El Salvador                  False           False           False
Guatemala                    False           False           False
Haiti                        False           False           False
Honduras                     False           False           False
Jamaica                      False           False           False
Mexico                        True            True            True
Nicaragua                    False           False           False
Panama                       False           False           False
Paraguay                     False           False           False
Peru                         False           False           False
Puerto Rico                   True            True            True
Trinidad and Tobago           True           False           False
United States                 True            True            True
Uruguay                      False           False            True
Venezuela                     True            True            True

                    gdpPercap_1997  gdpPercap_2002  gdpPercap_2007
country
Argentina                     True           False            True
Bolivia                      False           False           False
Brazil                       False           False           False
Canada                        True            True            True
Chile                         True            True            True
Colombia                     False           False           False
Costa Rica                   False           False           False
Cuba                         False           False           False
Dominican Republic           False           False           False
Ecuador                      False           False           False
El Salvador                  False           False           False
Guatemala                    False           False           False
Haiti                        False           False           False
Honduras                     False           False           False
Jamaica                      False           False           False
Mexico                        True            True            True
Nicaragua                    False           False           False
Panama                       False           False           False
Paraguay                     False           False           False
Peru                         False           False           False
Puerto Rico                   True            True            True
Trinidad and Tobago          False            True            True
United States                 True            True            True
Uruguay                       True           False           False
Venezuela                     True           False            True
```

In [5]:
```python
mask_higher.aggregate('sum',axis=1)
```

Out[5]:
```
country
Argentina              11
Bolivia                 0
Brazil                  1
Canada                 12
Chile                   3
Colombia                0
Costa Rica              0
Cuba                    4
Dominican Republic      0
Ecuador                 0
El Salvador             0
Guatemala               0
Haiti                   0
Honduras                0
Jamaica                 4
Mexico                  9
Nicaragua               0
Panama                  0
Paraguay                0
Peru                    2
Puerto Rico            10
Trinidad and Tobago     6
United States          12
Uruguay                 5
Venezuela              11
dtype: int64
```

Mask is True/False if data point is bigger than the mean for its column. `aggregate('sum', axis=1)` sums these as 1/0 values for each row. (`axis=0` would sum by column)

Define a "wealthy" country to be one whose GDP per capita is above the mean for more than 5 years in the data set

In [7]:
```python
wealthy = mask_higher.aggregate('sum',axis=1) > 5
print(wealthy)
```

```
country
Argentina               True
Bolivia                False
Brazil                 False
Canada                  True
Chile                  False
Colombia               False
Costa Rica             False
Cuba                   False
Dominican Republic     False
Ecuador                False
El Salvador            False
Guatemala              False
Haiti                  False
Honduras               False
Jamaica                False
Mexico                  True
Nicaragua              False
Panama                 False
Paraguay               False
Peru                   False
Puerto Rico             True
Trinidad and Tobago     True
United States           True
Uruguay                False
Venezuela               True
dtype: bool
```

Now can separately do statistics on the "wealthy" and "not wealthy" countries in the data set

In [8]:
```python
data_americas.groupby(wealthy).mean()
```

Out[8]:

	gdpPercap_1952	gdpPercap_1957	gdpPercap_1962	gdpPercap_1967	gdpPercap_1972	gdpPercap_1977	gdpPercap_1982	gdpPercap_1987	gdpPercap_1992	gdpPercap_1997	gdpPercap_200
False	2968.580475	3292.533177	3481.072065	3899.976473	4389.247147	4948.259836	5030.450172	5052.377184	5069.402016	5601.004531	5628.02931
True	6934.587894	8019.356593	8554.178512	10215.251554	11896.700691	13533.071588	13874.332014	14841.745316	15696.303408	17344.920001	18698.20001

This lets us easily see trends in values. For example, in 1952, wealthy mean is twice the "not wealthy" mean, but by 2007, this increases to three times

Can separate into more groups (either manually or computationally) and do more exotic statistics

`groupby` works for any series (strings, integer values, or True/False)

DataFrames are very useful when we're working with large amounts of data, but not always the easiest when we have small data sets

Natural way to store small amounts of data in Python is through a list, defined by `my_list = [item1, item2, ...]`

In [9]:
```python
primes = [2, 3, 5, 7]
print('primes is', primes)
print('length of primes is', len(primes))
```

```
primes is [2, 3, 5, 7]
length of primes is 4
```

`len(my_list)` gives the length of the list (how many items are in it)

In python, we index from zero and can select from or slice lists using indices, just like we did with strings

In [10]:
```python
print('Item 0 from primes is', primes[0])
print('Last item from primes is', primes[-1])
print('Slicing primes gives', primes[1:-1])
```

```
Item 0 from primes is 2
Last item from primes is 7
Slicing primes gives [3, 5]
```

One key difference between lists and strings is that lists are "mutable": we can replace a value by using its index

In [11]:
```python
primes[2] = 27
print('primes is ', primes)
```

```
primes is  [2, 3, 27, 7]
```

Contrast this with strings:

In [12]:
```python
my_string = "27 is not a prime number!"
my_string[1] = "5"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[12], line 2
      1 my_string = "27 is not a prime number!"
----> 2 my_string[1] = "5"

TypeError: 'str' object does not support item assignment
```

Fix our error:

In [13]:
```python
primes[2] = 5
print(primes)
```

```
[2, 3, 5, 7]
```

Use `append` to add a single item to a list, `extend` to concatenate lists:

In [14]:
```python
primes.append(11)
```

In [15]:
```python
print(primes)
```

```
[2, 3, 5, 7, 11]
```

In [16]:
```python
teen_primes = [13, 17, 19]
primes.extend(teen_primes)
print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19]
```

Be careful with append and extend: Cannot extend with a number:

In [17]:
```python
primes.extend(23)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 1
----> 1 primes.extend(23)

TypeError: 'int' object is not iterable
```

Can only extend with a list. Can append with a list, but it doesn't do what you want:

In [18]:
```python
primes.append([23, 29])
print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, [23, 29]]
```

Last entry in primes is now the list `[23, 29]`

In [19]:
```python
print(primes[8])
```

```
[23, 29]
```

Delete entries of a list using `del`

In [20]:
```python
del primes[8]
print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19]
```

Note differences between functions that act on lists, like `len(primes)`, python keywords (like `del`, `if`, and others that we'll see soon), and methods, like `append` and `extend`, that are associated with a list and act on its data.

Can see a list of methods on lists with

In [21]:
```python
help(list)
```

```
Help on class list in module builtins:

class list(object)
 |  list(iterable=(), /)
 |
 |  Built-in mutable sequence.
 |
 |  If no argument is given, the constructor creates a new empty list.
 |  The argument must be an iterable if specified.
 |
 |  Methods defined here:
 |
 |  __add__(self, value, /)
 |      Return self+value.
 |
 |  __contains__(self, key, /)
 |      Return bool(key in self).
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __getitem__(self, index, /)
 |      Return self[index].
 |
 |  __gt__(self, value, /)
 |      Return self>value.
 |
 |  __iadd__(self, value, /)
 |      Implement self+=value.
 |
 |  __imul__(self, value, /)
 |      Implement self*=value.
 |
 |  __init__(self, /, *args, **kwargs)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __iter__(self, /)
 |      Implement iter(self).
 |
 |  __le__(self, value, /)
 |      Return self<=value.
 |
 |  __len__(self, /)
 |      Return len(self).
 |
 |  __lt__(self, value, /)
 |      Return self<value.
 |
 |  __mul__(self, value, /)
 |      Return self*value.
 |
 |  __ne__(self, value, /)
 |      Return self!=value.
 |
 |  __repr__(self, /)
 |      Return repr(self).
 |
 |  __reversed__(self, /)
 |      Return a reverse iterator over the list.
 |
 |  __rmul__(self, value, /)
 |      Return value*self.
 |
 |  __setitem__(self, key, value, /)
 |      Set self[key] to value.
 |
 |  __sizeof__(self, /)
 |      Return the size of the list in memory, in bytes.
 |
 |  append(self, object, /)
 |      Append object to the end of the list.
 |
 |  clear(self, /)
 |      Remove all items from list.
 |
 |  copy(self, /)
 |      Return a shallow copy of the list.
 |
 |  count(self, value, /)
 |      Return number of occurrences of value.
 |
 |  extend(self, iterable, /)
 |      Extend list by appending elements from the iterable.
 |
 |  index(self, value, start=0, stop=9223372036854775807, /)
 |      Return first index of value.
 |
 |      Raises ValueError if the value is not present.
 |
 |  insert(self, index, object, /)
 |      Insert object before index.
 |
 |  pop(self, index=-1, /)
 |      Remove and return item at index (default last).
 |
 |      Raises IndexError if list is empty or index is out of range.
 |
 |  remove(self, value, /)
 |      Remove first occurrence of value.
 |
 |      Raises ValueError if the value is not present.
 |
 |  reverse(self, /)
 |      Reverse *IN PLACE*.
 |
 |  sort(self, /, *, key=None, reverse=False)
 |      Sort the list in ascending order and return None.
 |
 |      The sort is in-place (i.e. the list itself is modified) and stable (i.e. the
 |      order of two equal elements is maintained).
 |
 |      If a key function is given, apply it once to each list item and sort them,
 |      ascending or descending, according to their function values.
 |
 |      The reverse flag can be set to sort in descending order.
 |
 |  ----------------------------------------------------------------------
 |  Class methods defined here:
 |
 |  __class_getitem__(object, /)
 |      See PEP 585
 |
 |  ----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __new__(*args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate signature.
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __hash__ = None
```

Some of these make sense as regular functions:

In [22]:
```python
primes.index(5)
```

Out[22]: 2

Some of these don't really make sense as functions:

In [23]:
```python
primes.__mul__(2)
```

Out[23]: [2, 3, 5, 7, 11, 13, 17, 19, 2, 3, 5, 7, 11, 13, 17, 19]

This makes a list with 2 copies of primes (like string multiplication). Not interesting unto itself, but this is, in fact, the method called when we run

In [24]:
```python
primes*2
```

Out[24]: [2, 3, 5, 7, 11, 13, 17, 19, 2, 3, 5, 7, 11, 13, 17, 19]

We'll revisit these "dunder" methods a little later on

Two things to note about lists:

1. The empty list is a list.
2. Lists can be composed from different data types

In [25]:
```python
empty_list = []
len(empty_list)
```

Out[25]: 0

In [27]:
```python
mixed_list = ['1', 'dog', 3, 25]
print(mixed_list)
len(mixed_list)
```

```
['1', 'dog', 3, 25]
```

Out[27]: 4

An important difference between lists and some other data types is that assignment does not (automatically) copy data

In [28]:
```python
fun_numbers = primes
fun_numbers[0] = 27
print(fun_numbers)
```

```
[27, 3, 5, 7, 11, 13, 17, 19]
```

In [29]:
```python
print(primes)
```

```
[27, 3, 5, 7, 11, 13, 17, 19]
```

When we do assignment with lists, we just create a new name for the same data, not a new list with copied data. Instead, to tell Python to copy the data, we do

In [30]:
```python
new_primes = primes.copy()
new_primes[0] = 2
print(new_primes)
print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19]
[27, 3, 5, 7, 11, 13, 17, 19]
```

Note: This does what is called a "shallow copy", so if any list members are also lists, these are not copied, but still point to same location

In [31]:
```python
test = [0,[1,1],[2,2,2]]
```

In [32]:
```python
test2 = test.copy()
test2[0] = 3
print(test)
print(test2)
```

```
[0, [1, 1], [2, 2, 2]]
[3, [1, 1], [2, 2, 2]]
```

In [33]:
```python
print(test2[2])
```

```
[2, 2, 2]
```

In [34]:
```python
test2[2][1] = 4
print(test)
print(test2)
```

```
[0, [1, 1], [2, 4, 2]]
[3, [1, 1], [2, 4, 2]]
```

Similar to pointers in C: a list is an object and so we can either copy a reference to it or copy its values. For true copies of everything in a list, use `copy.deepcopy` from Python standard library

Can iterate over contents of a list by using `for` loops:

In [35]:
```python
for number in primes:
    print(number)
```

```
27
3
5
7
11
13
17
19
```

Use indentation in Python to indicate scope. Loop starts on line after :, ends when we go back to same indentation as original "for"

In [36]:
```python
for thing in test:
    print(thing)
    if type(thing) == list:
        for data in thing:
            print(data)
        print("that was a list")
    print("done with", thing)
```

```
0
done with 0
[1, 1]
1
1
that was a list
done with [1, 1]
[2, 4, 2]
2
4
2
that was a list
done with [2, 4, 2]
```

- indentation must be consistent. Often see 4 spaces
- Can name iteration variable whatever you want. Try to use meaningful names

In [37]:
```python
for prime in primes:
    prime_squared = prime**2
    print(prime, prime_squared)
```

```
27 729
3 9
5 25
7 49
11 121
13 169
17 289
19 361
```

For looping over regular sequence, use `range` statement:

In [38]:
```python
for number in range(5):
    print(number**2)
```

```
0
1
4
9
16
```

Note difference between:

In [39]:
```python
for prime in primes:
    print(prime**2)
```

```
729
9
25
49
121
169
289
361
```

In [41]:
```python
for index in range(len(primes)):
    prime = primes[index]
    print(prime**2)
```

```
729
9
25
49
121
169
289
361
```

First form is much preferred to the second, as it is more efficient, and also more readable

A common pattern in looping in Python is the accumulator:

In [42]:
```python
total = 0
for number in range(1,11):
    total = total + number
print(total)
```

```
55
```

In [43]:
```python
primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
total = 0
for prime in primes:
    total = total + prime
print(total)
```

```
129
```

Can also use while loops with similar syntax

In [45]:
```python
number1 = number2 = 0
while (number1 + number2) %2 == 0:
    number1 = int(input("Enter one number:"))
    number2 = int(input("Enter another number:"))
    print(number1+number2)
```

```
2
6
3
```

while loops execute until their condition evaluates to false

Some tasks naturally map onto `for` loops (e.g., accumulating a value over a list). Some naturally map onto while loops (keep accepting input until a condition is met). Some can easily be done in both ways

In [46]:
```python
index = 0
while primes[index+1] - primes[index] < 3:
    index = index + 1
print("First big gap in primes is ", [primes[index], primes[index+1]])
```

```
First big gap in primes is  [7, 11]
```

In [48]:
```python
for prime1, prime2 in zip(primes[:-1], primes[1:]):
    print("checking ", [prime1, prime2])
    if prime2 - prime1 > 2:
        print("First big gap in primes is ", [prime1, prime2])
        break
```

```
checking  [2, 3]
checking  [3, 5]
checking  [5, 7]
checking  [7, 11]
First big gap in primes is  [7, 11]
```

For loop version uses 3 ingredients:

1. `zip` lets us iterate over 2 (or more) lists of same length at same time
2. Nested if statement to check condition
3. `break` keyword that ends the loop if it is ever executed

In [50]:
```python
for prime1, prime2, prime3 in zip(primes[:-2], primes[1:-1], primes[2:]):
    print([prime1, prime2, prime3, prime1+prime2+prime3])
```

```
[2, 3, 5, 10]
[3, 5, 7, 15]
[5, 7, 11, 23]
[7, 11, 13, 31]
[11, 13, 17, 41]
[13, 17, 19, 49]
[17, 19, 23, 59]
[19, 23, 29, 71]
```

Key with if statements and while loops is control is determined by logical statements.

Can combine multiple statements with `and` or `or`

In [52]:
```python
masses = [5, 3, 7, 2, 9, 1]
speeds = [1, 9, 6, 10, 10 ,2]
for mass, speed in zip(masses, speeds):
    if (mass > 5 and speed > 5) or speed > 7:
        print("Fast particle: ", [mass, speed])
```

```
Fast particle:  [3, 9]
Fast particle:  [7, 6]
Fast particle:  [2, 10]
Fast particle:  [9, 10]
```

Can get nicer formatting using f-strings:

In [53]:
```python
for mass, speed in zip(masses, speeds):
    if (mass > 5 and speed > 5) or speed > 7:
        print(f"Fast particle with mass {mass} and speed {speed}")
```

```
Fast particle with mass 3 and speed 9
Fast particle with mass 7 and speed 6
Fast particle with mass 2 and speed 10
Fast particle with mass 9 and speed 10
```

General composition of an f-string is `f"test {variable} more text {another} ..."`

Lots of advanced features for special formatting:

In [54]:
```python
from math import pi
print("pi is ", pi)
print(f"pi is {pi}")
```

```
pi is  3.141592653589793
pi is 3.141592653589793
```

In [55]:
```python
print(f"pi to 6 digits is {pi:6f}")
```

```
pi to 6 digits is 3.141593
```

In [56]:
```python
print(f"pi to 6 digits in scientific notation is {pi:6e}")
```

```
pi to 6 digits in scientific notation is 3.141593e+00
```

Back to data sets:

Can use for loops in many ways to work with data. Example:

In [57]:
```python
import glob
for filename in glob.glob("./data/gapminder*.csv"):
    data = pd.read_csv(filename)
    print(filename, data['gdpPercap_2007'].max())
```

```
./data/gapminder_gdp_americas.csv 42951.65309
./data/gapminder_gdp_europe.csv 49357.19017
./data/gapminder_all.csv 49357.19017
./data/gapminder_gdp_oceania.csv 34435.36744
./data/gapminder_gdp_africa.csv 13206.48452
./data/gapminder_gdp_asia.csv 47306.98978
```

Use `glob` module from Python standard library to match filenames with a given pattern (think: output from a specific experiment or set of experiments, denoted by sensible file name convention).

So `glob.glob("<pattern>")` produces a list of all files that match `<pattern>` that we can loop over using a for loop, doing the same thing to each file of data

Note: if no matching pattern, the `glob` returns an empty list:

In [58]:
```python
print(glob.glob("./data/gapminder*.cvs"))
```

```
[]
```

In [60]:
```python
print(glob.glob("./data/gapminder*.csv"))
```

```
['./data/gapminder_gdp_americas.csv', './data/gapminder_gdp_europe.csv', './data/gapminder_all.csv', './data/gapminder_gdp_oceania.csv', './data/gapminder_gdp_africa.csv', './d
ata/gapminder_gdp_asia.csv']
```

What do we do with this data??? Probably want to plot it.

Visualizing our data sets helps us understand and explain them better

Most common library for creating plots in Python is `matplotlib`. `matplotlib` provides a lot of functionality, but most common plotting functions are in a sub-library called `matplotlib.pyplot`.

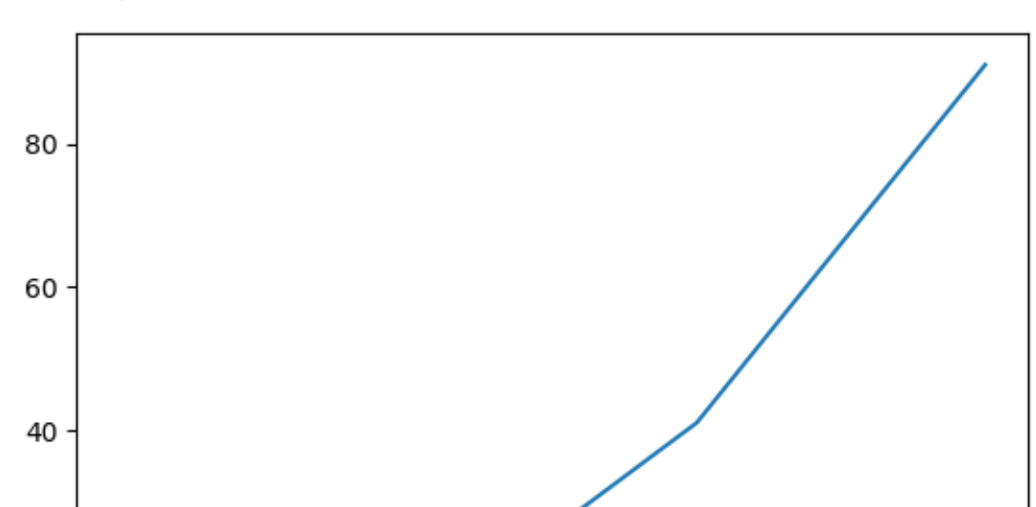
Other common plotting libraries, like `seaborn` build on `matplotlib`, so it's an important place to start even if you don't use it directly...

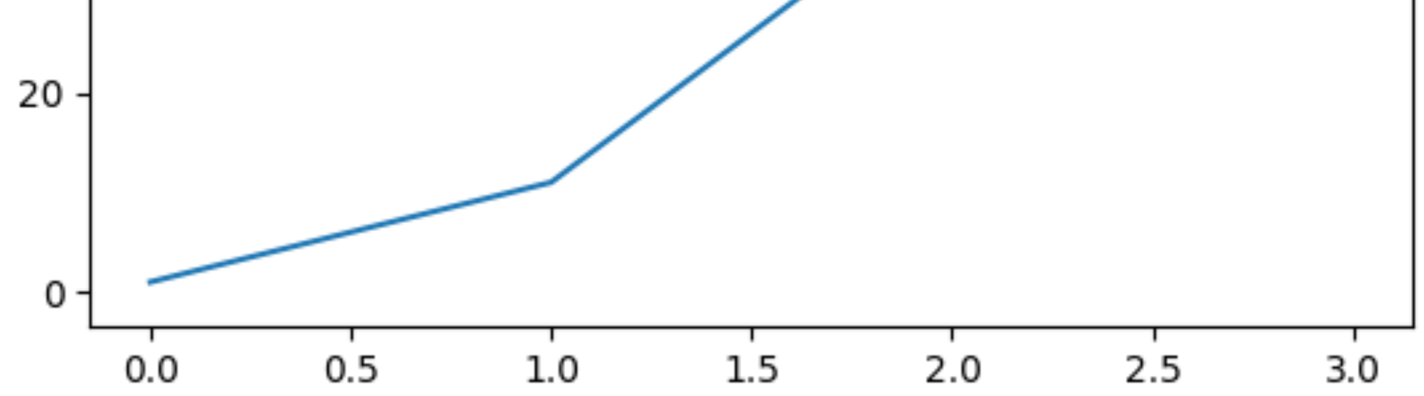
To install, activate venv and `pip install matplotlib`

Some simple examples:

In [61]:
```python
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
import matplotlib.pyplot as plt
plt.plot(times, positions)
```

Out[61]: [<matplotlib.lines.Line2D at 0x1234982d0>]

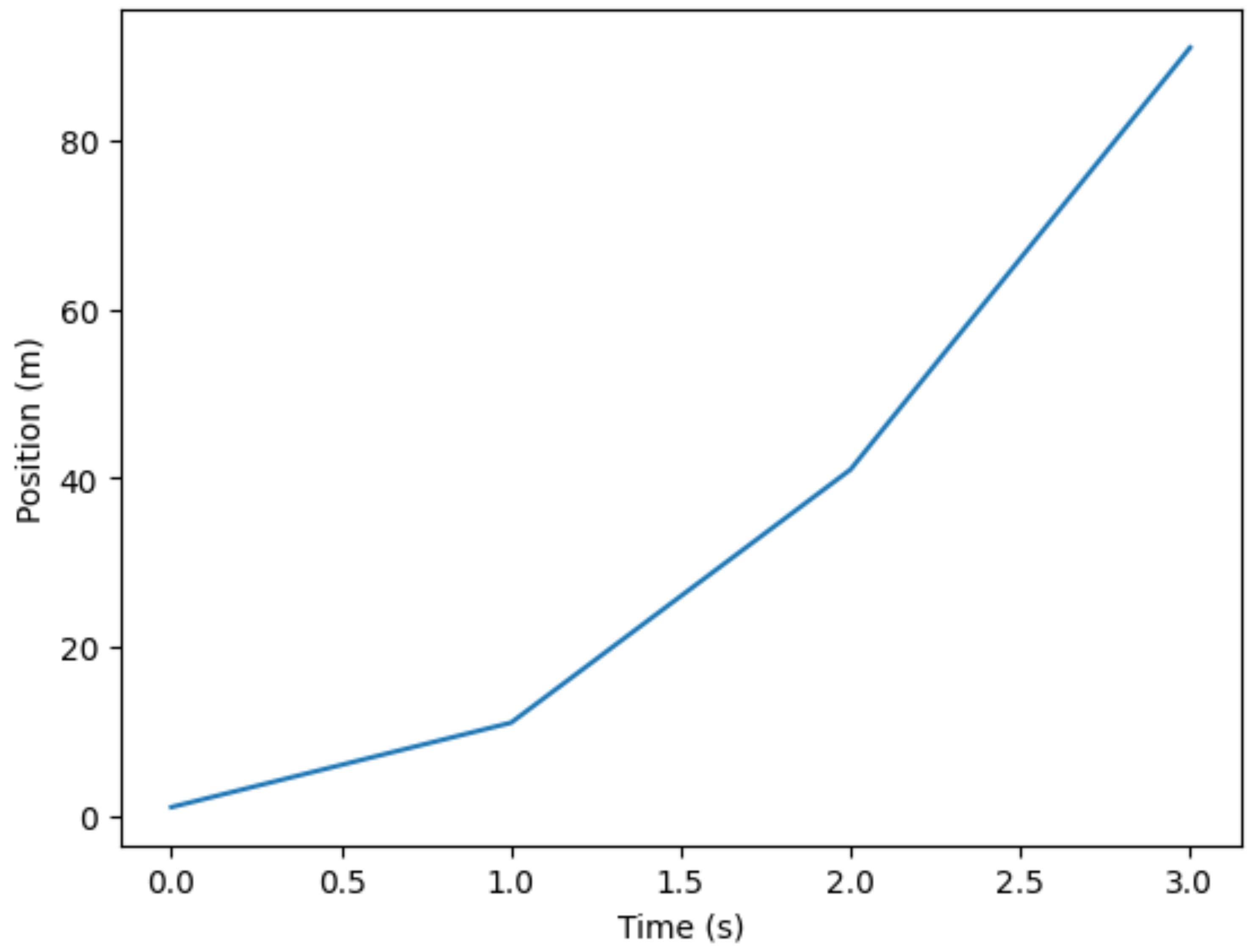




Is this any good? Not really. Need axis labels

```
In [62]: plt.plot(times, positions)
         plt.xlabel("Time (s)")
         plt.ylabel("Position (m)")
```

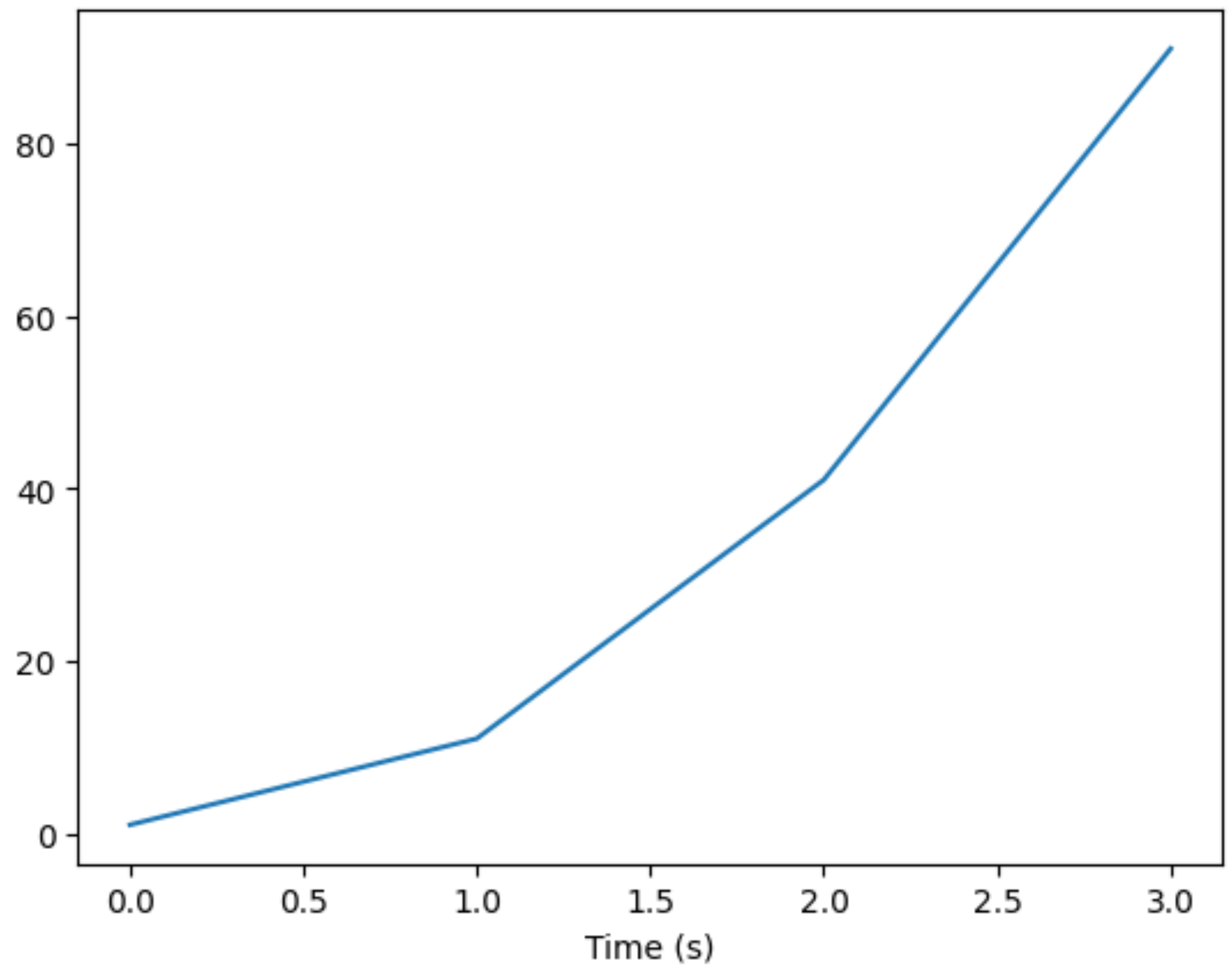
```
Out[62]: Text(0, 0.5, 'Position (m)')
```



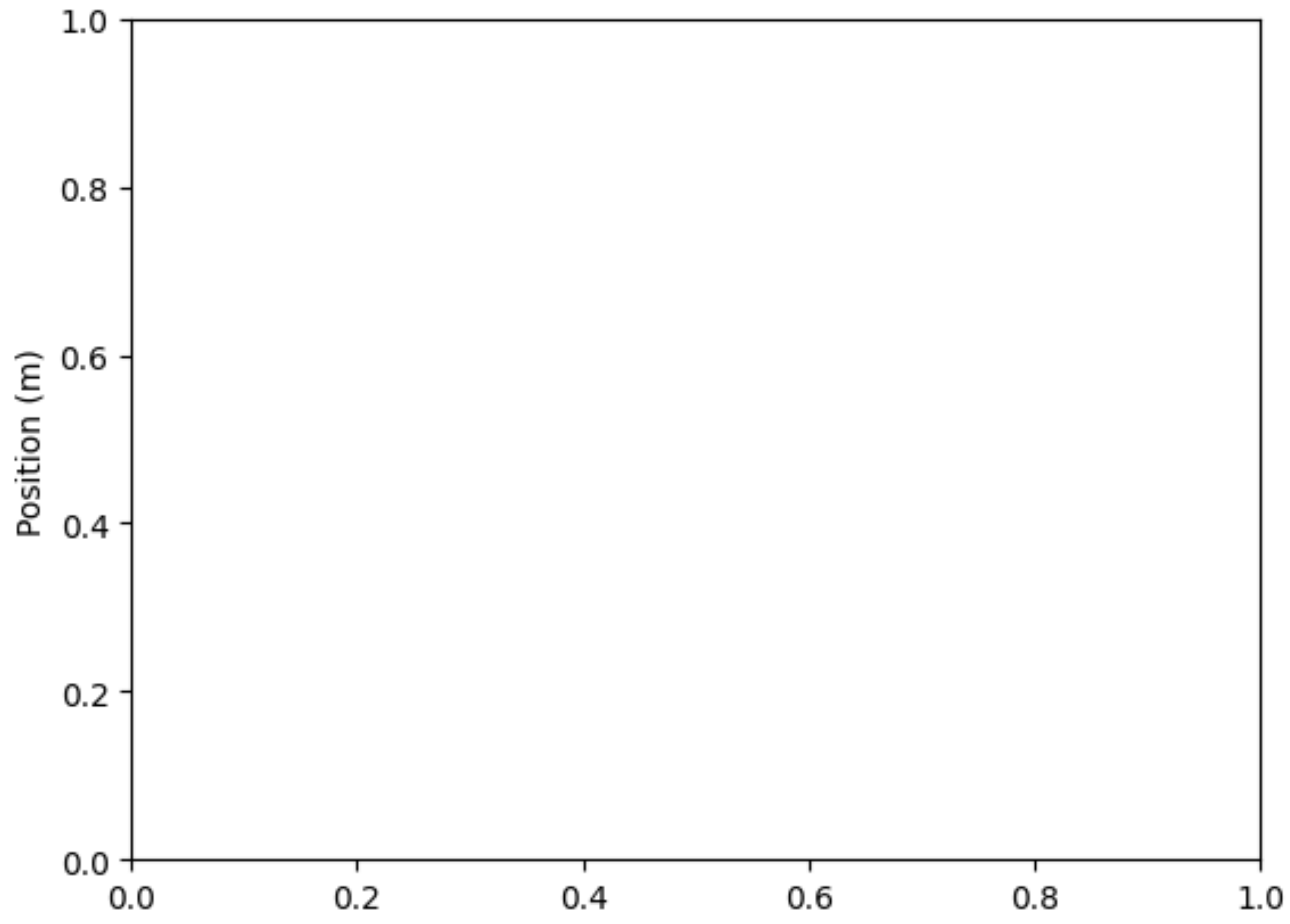
Note that in Jupyter notebook, must put all of our plot commands above the plot we wish to modify

```
In [63]: plt.plot(times, positions)
         plt.xlabel("Time (s)")
```

```
Out[63]: Text(0.5, 0, 'Time (s)')
```



```
In [64]: plt.ylabel("Position (m)")
```

```
Out[64]: Text(0, 0.5, 'Position (m)')
```